

AeroBook Premium

Complete Setup & Run Guide

Laravel 13 REST API | Microsoft SQL Server 2022 | Docker Compose | React 18 + Vite 5

Written for Windows 11 (PowerShell). Project folder: D:\AeroBook-Premium-main

Contents

1. Project overview and architecture
2. Prerequisites
3. First-time backend setup (Docker + SQL Server)
4. Frontend setup (React + Vite)
5. Using the application end to end
6. Connecting to the database (SSMS / sqlcmd)
7. Daily start and stop routine
8. Running the tests
9. VS Code shortcuts
10. Troubleshooting
11. Command cheat sheet

1. Project overview and architecture

AeroBook Premium is an airline reservation system. It is made of two separate apps that talk over HTTP:

- **backend/** - a Laravel 13 REST API (PHP 8.4) with Sanctum bearer-token auth. It runs inside Docker together with an Nginx web server and a Microsoft SQL Server 2022 database.
- **frontend/** - a React 18 single-page app built with Vite 5. It runs directly on your machine with Node.js and calls the API at `http://localhost:8000/api`.

Components and ports

Component	Container / process	Address	Purpose
SQL Server 2022	aerobook_sqlserver	localhost,1433	Database aerobook (data kept in a Docker volume)
DB init job	aerobook_sqlserver_init	-	Waits for SQL Server, creates the aerobook database, then exits
PHP-FPM (Laravel)	aerobook_app	internal :9000	Runs the API code
Nginx	aerobook_nginx	http://localhost:8000	Public entry point for the API
React frontend	npm run dev (Vite)	http://localhost:5173	The website you use in the browser

Request flow: **Browser** -> Vite dev server (5173) -> API calls to Nginx (8000) -> PHP-FPM (Laravel) -> SQL Server (1433).

Important folders

Path	What it contains
backend/app/Http/Controllers/Api	Auth, Flight, Booking, Airport and Admin controllers
backend/routes/api.php	All API routes
backend/database/migrations	Table definitions (run with <code>php artisan migrate</code>)
backend/database/seeder	Sample airports, aircraft, 12 flights and full seat maps
backend/database/sql	Raw SQL used by the controllers (seat map, booking locks, dashboard)
backend/docker-compose.yml	Definition of the four Docker services
frontend/src	React pages, components, API clients
docs/	Presentation, ERD diagram, improvement plan, and this guide

2. Prerequisites

Install the following once. Everything PHP-related (PHP, Composer, SQL Server drivers) is inside Docker, so you do **not** need PHP installed locally.

Software	Version	Required?	Notes
Docker Desktop	Latest	Yes	Use the WSL 2 backend. Give it at least 4 GB RAM (SQL Server needs ~2 GB).
Node.js (includes npm)	18 LTS or newer (20/22 recommended)	Yes	Needed for the Vite frontend.
A web browser	Chrome / Edge / Firefox	Yes	
SQL Server Management Studio	19+	Optional	To browse tables and run queries.
VS Code	Latest	Optional	The project ships ready-made tasks (section 9).

Verify the tools

Open **PowerShell** and run:

```
docker --version
docker compose version
node --version
npm --version
```

Note: Start Docker Desktop and wait until the whale icon says *Docker Desktop is running* before running any docker command. Otherwise you will see an error about the Docker engine / pipe not being found.

Free ports

Ports **1433**, **8000** and **5173** must be free. A locally installed SQL Server (e.g. SQL Express service) often occupies 1433 - see Troubleshooting if Docker reports *port is already allocated*.

```
netstat -ano | findstr ":1433 :8000 :5173"
```

No output means all three ports are free.

3. First-time backend setup (Docker + SQL Server)

Do these steps once on a new machine. All commands are run in PowerShell from the **backend** folder.

Step 3.1 - Go to the backend folder

```
cd D:\AeroBook-Premium-main\backend
```

Step 3.2 - Create the .env file

The project ships only `.env.example`. Laravel needs a real `.env`. The example is already configured for Docker (DB_HOST=sqlserver, user sa, database aerobook), so just copy it:

```
Copy-Item .env.example .env
```

Tip: Do not change DB_HOST to localhost when using Docker. Inside the Docker network, the database host name is **sqlserver**.

Step 3.3 - Build and start the containers

The first build downloads the PHP image, installs the Microsoft ODBC driver and sqlsrv extensions, and runs `composer install`. Expect 5-15 minutes on the first run; later starts take seconds.

```
docker compose up -d --build
```

Docker starts SQL Server, then runs **sqlserver-init**, which waits until SQL Server accepts connections and creates the **aerobook** database. Only after it succeeds are the app and nginx containers started.

Step 3.4 - Check the containers

```
docker compose ps -a
```

Expected result:

Service	Expected status
aerobook_sqlserver	Up
aerobook_sqlserver_init	Exited (0) - this is correct, it is a one-time job
aerobook_app	Up
aerobook_nginx	Up (0.0.0.0:8000->80/tcp)

Step 3.5 - Generate the application key

```
docker compose exec app php artisan key:generate
```

This writes APP_KEY into backend/.env. It is needed for encryption and sessions - without it the API returns *No application encryption key has been specified*.

Step 3.6 - Create the tables (migrations)

```
docker compose exec app php artisan migrate --force
```

This creates users, admins, passengers, airports, aircraft, flights, seats, bookings, payments, maintenance_logs, personal_access_tokens, sessions, cache and jobs tables in the **aerobook** database.

Step 3.7 - Load sample data (seeder)

```
docker compose exec app php artisan db:seed --force
```

The seeder adds:

- 4 airports: Dhaka (DAC), Chittagong (CGP), Sylhet (ZYL), Rajshahi (RJH).
- 2 aircraft: Boeing 737-800 and Airbus A320neo.
- 12 flights - one for every city pair - departing **tomorrow** from 08:00 (Asia/Dhaka), 30 minutes apart.
- A full 120-seat map per flight (rows 1-2 business, row 3 extra legroom, rows 12-13 exit rows, 20C/20D blocked).
- A test user test@example.com / password.

Note: Flights are dated relative to the day you run the seeder. If flights stop showing in search after a few days, simply run `db:seed --force` again - it updates the existing flights to tomorrow without duplicating them.

Step 3.8 - Create an administrator account

The admin panel uses a separate **admins** table, which the seeder does not fill. Create an admin with one command (change the email and password if you like):

```
docker compose exec app php artisan tinker --execute="DB::table('admins')->insert([
    'name'=>'Admin User', 'email'=>'admin@aerobook.test', 'password'=>bcrypt('password'),
    'role'=>'admin', 'created_at'=>now(), 'updated_at'=>now()]);"
```

Type or paste this as one single line - it is wrapped here only to fit the page.

You can now sign in at /admin - login with admin@aerobook.test / password.

Step 3.9 - Verify the API

Open these in a browser (or use `curl.exe`):

```
http://localhost:8000/api          -> {"application":"AeroBook API","status":"ok"}
http://localhost:8000/api/airports -> JSON list of the 4 airports
```

If both respond, the backend is fully working.

4. Frontend setup (React + Vite)

Open a **second** PowerShell window (leave Docker running) and run:

```
cd D:\AeroBook-Premium-main\frontend
npm install
npm run dev
```

`npm install` is only needed the first time (or after package.json changes). Vite prints something like:

```
VITE v5.x ready in 400 ms
-> Local:    http://localhost:5173/
-> Network:  http://192.168.x.x:5173/
```

Open **http://localhost:5173/** - you will be redirected to /home. Keep this terminal open; closing it stops the frontend. Press **Ctrl + C** to stop it.

Important: The backend only accepts browser requests (CORS) from ports **5173** and **5174** on localhost / 127.0.0.1. If Vite picks another port (5175, ...), the site will load but every API call fails. Free port 5173 or run `npm run dev -- --port 5173 --strictPort`.

Production build (optional)

```
npm run build      # output goes to frontend\dist
npm run preview    # serves the built files locally for a final check
```

5. Using the application end to end

Passenger flow

#	Page	What to do
1	/register	Create an account (name, email, passport, strong password). A passenger record is created together with the user.
2	/login	Sign in. The API returns a bearer token which the app stores in the browser.
3	/home	Search flights - e.g. Dhaka -> Chittagong on tomorrow's date (seeded flights).
4	/flights/:id/seats	Pick seats on the colour-coded map: green = available, blue = selected, amber B = booked, grey X = sold, dark = blocked. It refreshes every 8 seconds.
5	Payment	Complete the payment form. Seats are locked in a database transaction so two people can never book the same seat.
6	Confirmation / /my-bookings	View the confirmation and all your bookings.
7	/profile	Update name, phone, etc.

Admin flow

- Go to `http://localhost:5173/admin` - login and sign in with the admin created in step 3.8.
- The dashboard at `/admin` shows totals for flights, bookings, revenue and fleet health (data from `GET /api/admin/dashboard`).

Password reset

Mail is configured with `MAIL_MAILER=log`, so no real email is sent. After you request a code on `/reset-password`, read the 6-digit code from the Laravel log:

```
Get-Content D:\AeroBook-Premium-main\backend\storage\logs\laravel.log -Tail 30
```

Main API endpoints (base `http://localhost:8000/api`)

Method	Endpoint	Auth
GET	/airports, /flights/search, /flights/{id}/seatmap	Public
POST	/auth/register, /auth/login, /auth/admin-login	Public (rate limited)
POST	/auth/password/request-code, /auth/password/reset	Public (rate limited)
GET / PUT / POST	/auth/me, /auth/profile, /auth/logout	Bearer token
GET / POST	/bookings, /bookings/{id}	Bearer token
GET	/admin/dashboard	Bearer token + admin role

6. Connecting to the database

SQL Server Management Studio (SSMS)

Field	Value
Server type	Database Engine
Server name	localhost,1433 (comma, not colon)
Authentication	SQL Server Authentication
Login	sa
Password	AeroBook_SqlServer_2026!
Encryption / certificate	Tick Trust server certificate
Database	aerobook (Options >> Connection Properties, or pick it after connecting)

Registered users appear in **aerobook > Tables > dbo.users**, and their passenger rows in **dbo.passengers**. Bookings, seats and payments are in the tables with those names.

sqlcmd from inside the container (no SSMS needed)

(one single line)

```
docker compose exec sqlserver /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P  
"AeroBook_SqlServer_2026!" -C -d aerobook -Q "SELECT id, name, email FROM users"
```

About aerobook_database.sql: backend/database/aerobook_database.sql is an older, stand-alone script that builds the schema with raw SQL. Use migrations + seeder (section 3) as the main path; do not run both on the same database, because the raw script's sample data assumes its own IDs.

7. Daily start and stop routine

After the first-time setup you only need the following.

Start

```
# 1. Open Docker Desktop and wait until it is running

# 2. PowerShell window #1 - backend
cd D:\AeroBook-Premium-main\backend
docker compose up -d
docker compose exec app php artisan migrate --force # only if new migrations were added

# 3. PowerShell window #2 - frontend
cd D:\AeroBook-Premium-main\frontend
npm run dev
```

Then open <http://localhost:5173/>.

Stop

```
# frontend window: press Ctrl + C

# backend
cd D:\AeroBook-Premium-main\backend
docker compose down
```

`docker compose down` keeps your data - it lives in the **aerobook_sqlserver_data** volume. Only `docker compose down -v` deletes it.

After changing backend code

The backend folder is mounted into the container, so PHP changes apply immediately (no rebuild).
Rebuild only when you change the **Dockerfile** or **composer.json**:

```
docker compose up -d --build
docker compose exec app composer install
```

8. Running the tests

Backend (PHPUnit - uses an in-memory SQLite DB, so it does not touch your data):

```
docker compose exec app php artisan test
```

Frontend (Node's built-in test runner - seat map and search helpers):

```
cd D:\AeroBook-Premium-main\frontend  
npm test
```

9. VS Code shortcuts

Open the **D:\AeroBook-Premium-main** folder in VS Code. The project includes:

- **Terminal > Run Task > Backend: docker compose up** - starts the containers and runs migrations.
- **Terminal > Run Task > Frontend: npm run dev** - starts Vite on 127.0.0.1:5173.
- **Run and Debug > AeroBook: run in Chrome (F5)** - starts the frontend task and opens <http://127.0.0.1:5173/home> in Chrome with the debugger attached.

10. Troubleshooting

Problem	Cause	Fix
<i>error during connect ... docker_engine</i>	Docker Desktop is not running.	Start Docker Desktop and wait until it is ready.
<i>Bind for 0.0.0.0:1433 failed: port is already allocated</i>	A local SQL Server service already uses 1433.	Stop it: services.msc -> SQL Server (MSSQLSERVER / SQLEXPRESS) -> Stop. Or change the left side of "1433:1433" in docker-compose.yml (e.g. 1434:1433) and use localhost,1434 in SSMS.
Port 8000 already allocated	Another app (e.g. php artisan serve) uses 8000.	Stop that app, or change "8000:80" and API_BASE_URL in frontend/src/api/axiosClient.js.
sqlserver container keeps restarting / exits	Too little memory for SQL Server.	Docker Desktop -> Settings -> Resources: give 4 GB+. Check docker compose logs sqlserver.
app/nginx never start; sqlserver-init not Exited (0)	SQL Server not ready or wrong password.	docker compose logs sqlserver-init. Then docker compose up -d again.
<i>No application encryption key has been specified</i>	APP_KEY is empty.	docker compose exec app php artisan key:generate
<i>vendor/autoload.php: Failed to open stream</i>	Composer packages missing in the vendor volume.	docker compose exec app composer install
HTTP 500 / DB login failed	.env missing or DB_HOST not 'sqlserver'.	Recreate .env from .env.example (step 3.2) and run key:generate again.
<i>There is already an object named 'users'</i>	Tables created by another method (e.g. the raw SQL script).	Reset the DB: php artisan migrate:fresh --seed --force (deletes all data).
Frontend shows 'Network Error' or CORS error	Backend not running, or Vite is not on 5173/5174.	Check http://localhost:8000/api and run Vite on port 5173.
Redirected to /login unexpectedly	The token expired or was rejected (HTTP 401).	Sign in again.
Flight search returns nothing	Seeded flights are in the past or wrong date searched.	Search tomorrow's date, or re-run db:seed --force.
<i>Too Many Attempts</i> on login/register	Built-in rate limit (10-20 per minute).	Wait one minute.
'npm' is not recognized	Node.js not installed / not on PATH.	Install Node.js LTS and reopen PowerShell.

Viewing logs

```
docker compose logs -f app          # PHP-FPM output
docker compose logs -f nginx        # HTTP access / errors
docker compose logs -f sqlserver    # database engine
Get-Content backend\storage\logs\laravel.log -Tail 50 # Laravel application errors
```

Full reset (start completely from scratch)

```
cd D:\AeroBook-Premium-main\backend
docker compose down -v          # WARNING: deletes the database and vendor volumes
docker compose up -d --build
docker compose exec app php artisan key:generate
docker compose exec app php artisan migrate --force
docker compose exec app php artisan db:seed --force
```

Then repeat step 3.8 to recreate the admin account.

11. Command cheat sheet

Task	Command (run in backend\ unless noted)
Create .env (first time)	Copy-Item .env.example .env
Build + start backend	docker compose up -d --build
Start backend (normal)	docker compose up -d
Container status	docker compose ps -a
Generate app key	docker compose exec app php artisan key:generate
Run migrations	docker compose exec app php artisan migrate --force
Seed sample data	docker compose exec app php artisan db:seed --force
Wipe + rebuild tables	docker compose exec app php artisan migrate:fresh --seed --force
List API routes	docker compose exec app php artisan route:list
Clear cached config	docker compose exec app php artisan optimize:clear
Laravel shell	docker compose exec app php artisan tinker
Backend tests	docker compose exec app php artisan test
Stop backend	docker compose down
Install frontend deps (frontend\)	npm install
Run frontend (frontend\)	npm run dev
Frontend tests (frontend\)	npm test
Production build (frontend\)	npm run build

Default credentials

Account	Username / email	Password
SQL Server	sa	AeroBook_SqlServer_2026!
Seeded user	test@example.com	password
Admin (created in step 3.8)	admin@aerobook.test	password

Security: These are development credentials only. Change the SA password (docker-compose.yml and .env), set APP_DEBUG=false and use a real mail driver before deploying anywhere public.